

Comparing Three Numerical Methods for Planar Two-Link Arm Motion

CSE402: Numerical Analysis, Simulation and Modeling Sessional · Section C-2

2105161 — Sadia Naushin

2105174 — Nujhat Sadia

2105163 — Dibbo Chowdhury

2105176 — Navin Nawar

2105173 — Sunanda Chowdhury

2105179 — Arpa Das

Problem Statement & Scope

The Problem

Move a planar two-link arm from pose A to pose B :

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau, \quad t \in [0, T]$$

$$q(0) = q_A, \quad q(T) = q_B$$

$$q_{\min} \leq q(t) \leq q_{\max}, \quad |\tau| \leq \tau_{\max}$$

Objective

$$\min \int_0^T (\|\tau\|^2 + w_{\dot{q}} \|\dot{q}\|^2) dt$$

Approach

- ▶ **Cross-domain**: numerical tools applied to robot motion
- ▶ **Comparative**: same model, three methods
- ▶ **Not** a new algorithm

Scope

- ▶ Simulation only (Python)
- ▶ Two-link planar arm
- ▶ No hardware; no high-DoF pipeline

The Robot Model

State & Control

State: $x = [\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]^\top \in \mathbb{R}^4$

Control: $u = \tau = [\tau_1, \tau_2]^\top \in \mathbb{R}^2$

State-space form:

$$\dot{x} = f(x, u) = \begin{bmatrix} \dot{q} \\ M^{-1}(u - C\dot{q} - g) \end{bmatrix}$$

Manipulator Equations

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau$$

- ▶ $M(q)$ — Inertia matrix (2×2 , SPD)
- ▶ $C(q, \dot{q})\dot{q}$ — Coriolis & centrifugal
- ▶ $g(q)$ — Gravity torque vector

Parameters

Link lengths l_1, l_2 ; masses m_1, m_2 ; inertias I_1, I_2 ;
 $g_0 = 9.81 \text{ m/s}^2$

Three Numerical Methods at a Glance

	Shooting (RK4)	Trapezoidal	Hermite-Simpson
Decision vars	Controls: $2N$	States+Ctrl: $6(N+1)$	+midpoints: $6(2N+1)$
Physics	RK4 in loop	Defect= 0 (trapz)	Defect= 0 (Simpson)
Accuracy	$\mathcal{O}(h^4)$ /step	$\mathcal{O}(h^2)$ global	$\mathcal{O}(h^4)$ global
Speed ($N=20$)	6.37 s	3.14 s ← FASTEST	12.26 s
$\int \ \tau\ ^2 dt$	282.95	223.41	217.79 ← LOWEST
DOP853 drift	1.77 m ×	0.071 m ✓	0.280 m ✓

Base paper: Cardona-Ortiz & Arechavaleta (2025) AMCS 35(4) 577–589 Method recipes: Kelly, SIAM Review 2017

Algorithm

1. Start from $x_0 = x_A$
2. Apply torque u_k for step h
3. RK4 integration $\rightarrow x_{k+1}$
4. Repeat: $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_N$
5. SLSQP adjusts $u_0, \dots, u_{N-1}$
6. Minimise $\int \tau^2 dt$, s.t. $x_N \approx x_B$

RK4 Formula

$$x_{k+1} = x_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \text{ where}$$

$$k_i = f(x_k + \dots, u_k)$$

Known Weakness

Optimizer reports EE error = 0 m, but replaying through DOP853 gives drift = **1.77 m**.

Long chain $u_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_N$: small changes early grow along the horizon. (Part of the drift comes from replaying piecewise-constant torques with linear interpolation.)

Decision Vector

$$z = [u_0^\top, \dots, u_{N-1}^\top]^\top \in \mathbb{R}^{2N}$$

Only **controls** are unknowns; states are

Core Idea

Split $[0, T]$ into N intervals. Every x_k and u_k is an **unknown** ($6(N+1)$ variables). Physics is enforced between neighbouring nodes, all at once.

Defect Constraint (Physics)

$$\Delta_k = x_{k+1} - x_k - \frac{h}{2} [f_k + f_{k+1}] = 0$$

Trapezoidal rule: $\int_{t_k}^{t_{k+1}} f \, dt \approx \frac{h}{2} [f_k + f_{k+1}]$
 $k = 0, \dots, N-1 \Rightarrow 4N$ equations

Constraints & Solver

- ▶ Boundary: $x_0 = x_A, x_N = x_B$
- ▶ Box bounds: $|\tau| \leq \tau_{\max}$
- ▶ Objective: $\frac{h}{2} [L_0 + 2L_1 + \dots + L_N]$
- ▶ Solver: SLSQP

Baseline Result ($N=20$)

Solve: 3.14 s (fastest of the three)
 DOP853 final drift: 0.071 m

S1: Fix the Start and End States Directly

Problem

The start and end pose are already **known exactly**, yet the old code spent 8 equality constraints (out of $4N$) forcing the optimizer to *discover* them.

Fix

Set the lower and upper **bound** of those 8 variables to the same known value. The optimizer can no longer move them — no equations needed at all.

Honest Result

A warm-up, not the headline: 8 fewer equations out of ~ 124 barely moves the needle.

	Time ($N=40$)	Iterations
Original	21.54 s	46
+ S1	20.32 s	46

Speedup: $1.1\times$. Same cost as baseline (to within 1.4×10^{-7} relative) — correctness is unchanged, only how the answer is asked for.

S2: Compute the Physics for Every Point at Once

Problem

The original code looped over all $N+1$ nodes in **Python**, calling the dynamics one node at a time. Python loops carry overhead on every iteration.

Fix

Vectorize: compute $f(x, u)$ for *all* nodes in one NumPy call. Also replace the general matrix solve with the **closed-form** 2×2 inverse of $M(q)$.

Verified

Matches the old node-by-node loop to $\text{rtol}=10^{-10}$ on 1000 random states — same math, just faster bookkeeping.

	Time ($N=40$)	Iterations
+ S1	20.32 s	46
+ S2	7.05 s	46

Biggest single jump of all five changes ($\approx 2.9\times$ on its own). Tells us the original slowdown was Python overhead, not hard math.

S3: Sparse (“Colored”) Jacobian — the Main Contribution

Problem

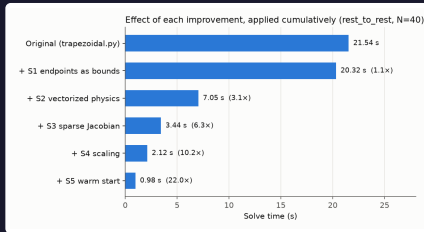
Without help, SLSQP nudges variables **one at a time** to learn the Jacobian: $6(N+1)=246$ full re-evaluations per iteration at $N=40$.

Key Insight

Defect k only touches nodes $k, k+1$. Non-neighbouring nodes can be nudged **together** without confusing their effects — a graph-colouring problem.

Fix

Greedy colouring always finds **12 groups** ($6 \text{ vars} \times \{\text{even, odd}\} \text{ nodes}$), for any N . Exact gradient supplied for the objective too.



$7.05 \text{ s} \rightarrow 3.44 \text{ s}$ ($\approx 2.0\times$ on its own). The group count stays 12 no matter how large N gets.

S4: Rescale the Numbers (Variable + Objective Scaling)

Problem

Angles ~ 1 , speeds up to $\dot{q}_{\max} \approx 15$, torques up to $\tau_{\max} \approx 30$: wildly different scales confuse the optimizer's step sizes.

Fix

Rescale every variable, $y = z/s$, to be $\mathcal{O}(1)$. **Also** divide the objective by $h\tau_{\max}^2$ so its curvature matches.

Applied Halfway

Scaling variables **alone**: $46 \rightarrow 115$ iterations (worse!). Scaling **both**: stays at 46.

	Time ($N=40$)	Iterations
+ S3	3.44 s	46
+ S4	2.12 s	46

Lesson: a change applied only halfway can backfire. On `high_speed`, scaling both also cuts iterations from 111 to 56.

S5: Warm Start — Solve an Easy Version First

Problem

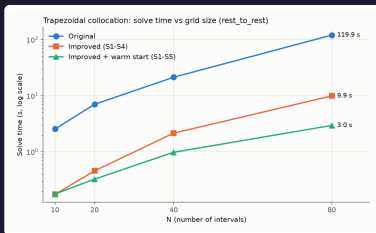
The default guess is a straight line with **zero velocity everywhere** — physically impossible, so the optimizer starts far from a good answer.

Fix

Solve a **coarse** grid first (e.g. $N=10$), stretch that answer onto a finer grid by interpolation, repeat up to the target N .

Fairness Rule

Reported time/iterations are **totals across all levels** — coarse solves included, so nothing is hidden.



2.12 s $\rightarrow$ **0.98 s** ($N=80$: **40.6 $\times$** faster than original). **Bonus:** on reversal/obstacle, the coarse solve reliably picks the good valley, fixing a different-answer issue seen with S1–S4 alone.

A1: The Motion Between Nodes

Problem

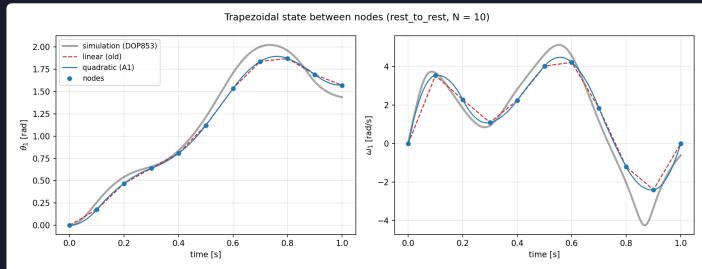
The optimizer only gives values **at nodes**. In between, the old code joined nodes with **straight lines**.

Why wrong

Trapezoid assumes $f = \dot{x}$ is linear, so x must be a **parabola** (like $\frac{1}{2}at^2$).

Fix (Kelly 2017)

$$x(t_k + \sigma) = x_k + f_k \sigma + \frac{\sigma^2}{2h} (f_{k+1} - f_k)$$



Local error at $N=40$: $0.685 \rightarrow$ **0.256** ($2.7\times$ lower); A2 and A4 build on this formula.

A2: Proving the Error is $\mathcal{O}(h^2)$

Problem

" $\mathcal{O}(h^2)$ " was only claimed; results existed for one N only.

Theory

$$N \times 2 \Rightarrow h \div 2 \Rightarrow \text{error} \div 2^2 = \div 4$$

N	h	rest_to_rest		high_speed	
		Error	Drop	Error	Drop
20	h_0	3.108	—	1.239	—
40	$h_0/2$	0.687	4.5×	0.317	3.9×
80	$h_0/4$	0.172	4.0×	0.080	4.0×
160	$h_0/8$	0.044	3.9×	0.020	3.9×

A3: Checking the Whole Arm, Not Just the Hand

Problem

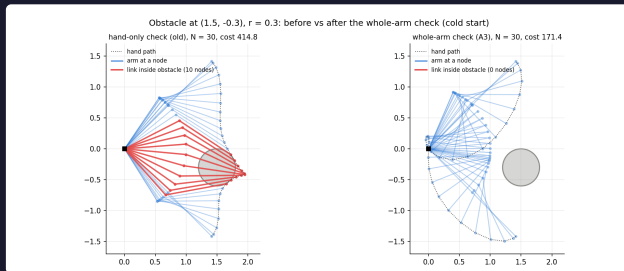
Only the **hand** was kept outside the obstacle. Links passed through it while the solver said “success”.

Fix

Each link is a **segment**: keep its closest point to the centre (projection, clamped to $[0, 1]$) outside r .

Found

The old obstacle is **infeasible** for the whole arm (link 1 passes 0.2 m from its centre), so we moved it.



cost 414.8 $\rightarrow$ 171.4: the arm folds its elbow instead.

A4: Checking Between the Nodes

Problem

Obstacle checked **only at nodes**: the path between two safe nodes cut through it ("corner cutting").

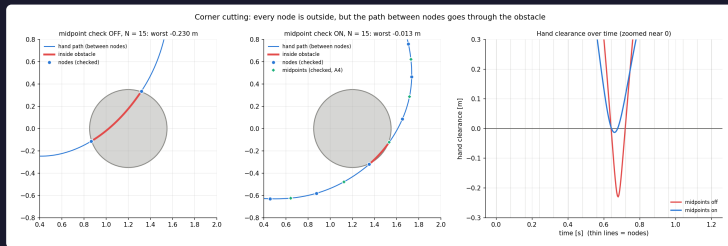
Fix

Also check each interval's **midpoint**, from the A1 parabola at $\sigma = h/2$:

$$x_{\text{mid}} = x_k + \frac{h}{8}(3f_k + f_{k+1})$$

Honest limits

Safety costs effort (+24%).
Fails at $N=8$: halving the gap is not a guarantee.



Key Idea

Add one extra point in the **middle** of every time step, and fit a smooth curve through it. This makes the accuracy much better for the same number of time steps.

Midpoint & Physics Check

Midpoint value:

$$x_{k+\frac{1}{2}} = \frac{x_k + x_{k+1}}{2} + \frac{h}{8} [f_k - f_{k+1}]$$

Physics check (Simpson's rule):

$$\Delta_k = x_{k+1} - x_k - \frac{h}{6} [f_k + 4f_{k+\frac{1}{2}} + f_{k+1}] = 0$$

Compared to Trapezoidal

- ✓ Much better accuracy for the same N
- ✓ Uses the least torque of all three: 217.79
- ✓ Smoother torque over time
- ✗ More unknowns per step
- ✗ Slowest to solve: 12.26 s

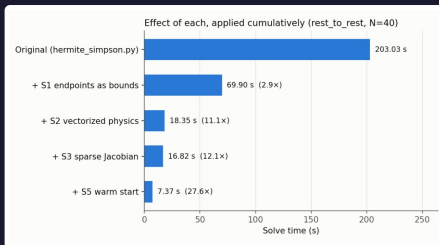
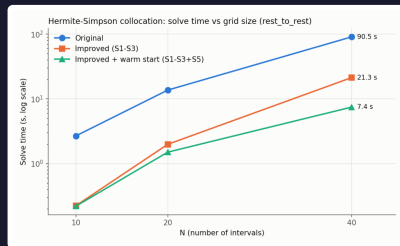
Same Changes, Reused

Reused directly from Trapezoidal:

- 1 Lock start/end directly
- 2 Compute physics for every point at once
- 3 Recompute derivatives smartly (18 checks, not hundreds)
- 5 Solve an easy, coarse version first

Result

4.3–13.3× faster, same answer every time, across every N tested (10, 20, 40).



Does the Speedup Cost Us Accuracy?

Short Answer: No

Real-physics drift barely changes between the original and the sped-up solver, at every grid size we tried:

- ▶ $N=10$: **1.408 m** for all three versions
- ▶ $N=20$: 0.2805 m vs 0.2808 m
- ▶ $N=40$: 0.0373 m vs **0.0304 m** (slightly better)

Since the optimizer finds the same trajectory either way, the accuracy comes along for free. The real lever for accuracy is N , not which version of the solver you run.

One Honest Exception

On the hardest, most oscillating task (*reversal*), the warm-start version did not just fail to help, it actively **failed**: slower than solving directly (61 s vs 8.8 s), 632 iterations instead of 68, and a constraint error several orders of magnitude worse than every other result ($5.6e-04$ vs. a typical $1e-9$).

Takeaway: warm start is not free money. Always check success before trusting a warm-started answer.

Benchmark Results — Rest-to-Rest Scenario

$N=20$, $T=1.0$ s, $\tau_{\max}=30$ Nm. All three methods converged ✓

Metric	Shooting (RK4)	Trapezoidal	Hermite-Simpson
Solve time	6.371 s	3.143 s	12.264 s
Iterations	53	39	54
$\int \ \tau\ ^2 dt$ (Nm ² s)	282.95	223.41	217.79
Peak torque (Nm)	25.14	26.53	28.61
EE error (opt.)	0.000 m	0.000 m	0.000 m
DOP853 max drift	1.901 m	0.120 m	0.280 m
DOP853 final drift	1.771 m	0.071 m	0.280 m

Key Insights from Our Experiments

Speed Winner: Trapezoidal

3.14 s — $2\times$ faster than Shooting, $4\times$ faster than Hermite-Simpson: fewer variables than Hermite-Simpson and no simulation inside the loop. With S1–S5: up to $56\times$ faster again.

Shooting Open-Loop Drift

Optimizer claims 0 error, but DOP853 gives 1.77 m drift. The long $u \rightarrow x$ chain makes the end state very sensitive to early torques.

Hermite-Simpson: Lowest Effort

$217.79\text{ Nm}^2\text{s} = 23\%$ less than Shooting. Cubic splines discover smoother, more energy-efficient torques.

Hard Tasks: Collocation Wins

Under-torqued swing-up: Shooting **FAILS** (exponential sensitivity blowup). Both collocation methods succeed — states anchored at every node.

When to Use Which Method?

Trapezoidal — Fast Real-Time Planning

Fewest iterations, fastest solve, robust convergence on standard tasks.

Hermite-Simpson — High Accuracy / Smooth Motion

$\mathcal{O}(h^4)$ global accuracy, lowest effort, cubic spline trajectories — fewer nodes needed.

Any Collocation — Unstable / Swing-Up Tasks

Shooting fails (exponential sensitivity). Collocation anchors state at every node independently.

Shooting (RK4) — Simple Short-Horizon Tasks

Low variable count, easy to implement — only when horizon is short & system is stable.

Cardona-Ortiz & Arechavaleta (2025), *AMCS* 35(4) 577–589 Kelly, *SIAM Review* 2017

Thank You!

Questions & Discussion Welcome

- ▶ Shooting: simple but drifts on long horizons
 - ▶ Trapezoidal: fastest, most practical
- ▶ Improved trapezoidal: up to $56\times$ faster, $\mathcal{O}(h^2)$ verified, collision-free
 - ▶ Hermite-Simpson: most accurate, lowest energy
 - ▶ Collocation always wins on hard tasks

Sadia Naushin · Dibbo Chowdhury · Sunanda Chowdhury · Nujhat Sadia · Navin Nawar · Arpa
Das